Sprint 15 · Reto «El mejor modelo, demostrado»

Ficha completa del reto, para consultar. Para trabajar con ella abierta, usad el «reto en una página» del sprint (carpeta `1_Reto_en_una_pagina`).

- Volver al README del sprint · README del sprint en PDF · ecofriendly
- **Esta ficha en PDF:** a color · ecofriendly, para imprimir
- Si es vuestro primer reto: Cómo se trabaja un reto.

RETO Ficha del reto

1 Qué os pide el cliente

Qué os pide el cliente. El cliente (el docente) quiere saber **qué modelo recomendáis para vuestro proyecto y por qué**. No le basta con «el que tiene el número más alto»: quiere saber si la mejora frente al *baseline* (el modelo más simple, que sirve de referencia) **compensa el coste y la complejidad**, si el modelo **funciona con datos que no ha visto** y si vuestros resultados **se pueden repetir** otro día y en otro ordenador. Este sprint es el corazón técnico del proyecto, pero lo que se valora **no es probar muchos modelos**: es **probar con orden, anotar cada prueba y elegir con honestidad**.

Cómo se trabaja. Partís de lo que ya tenéis del Sprint 14 (datos, análisis y *baseline*) y de las herramientas del cuaderno de práctica: una función que fija el azar, otra que anota cada prueba en un fichero y la validación cruzada (evaluar varias veces con trozos distintos de los datos). Vosotros decidís **antes** qué modelos probáis y por qué (como mínimo, tres modelos clásicos y uno neuronal, o un asistente LLM/RAG si vuestro brief es el 3), los comparáis **con la misma partición**, **elegís el candidato sin mirar el test** y solo después lo evaluáis en el test **una única vez**. Son tres jornadas: **470 minutos de reto** (130 el día 1, 200 el día 2 y 140 el día 3) más 60 de review y retro. El equipo decide hasta dónde llega: mínimo, estándar o extra.

2 · Recursos del reto

Recurso	Para qué lo usaréis	Dónde está
Python 3.11 y las bibliotecas del proyecto (`pandas`, `scikit-learn`, `matplotlib`; la de redes que usasteis en el Sprint 10)	Entrenar, comparar y guardar los modelos	Colab, o el entorno del equipo con las versiones de vuestro `requirements.txt`
Cuaderno del sprint `S15_01_seguimiento_experimentos`	Copiar las funciones `fijar_semillas`, `registrar` y `experimento`, y ver cómo se mide con validación cruzada, cómo se detecta una fuga y cómo se evalúa el test una vez	Colab (50 minutos el día 1). MLflow es **opcional**
Plantilla del registro `registro_experimentos_plantilla.csv`	Solo trae la cabecera de las columnas; es la base de vuestro `registro.csv`	En GitHub: *Raw* → *Download* o copiar el texto
Píldora y apuntes presentación S15.1 · apuntes	El porqué de cada regla (semillas, fuga, media ± desviación, test una vez), con ejemplos numéricos	En el repositorio del curso
Vuestro trabajo del Sprint 14	Los datos, el *baseline* y el cuaderno de partida	El repositorio del equipo (`ifcd107-equipo-…`)

Recurso	Para qué lo usaréis	Dónde está
Cómo entregar en GitHub	Subir los ficheros a la carpeta `sprint-15/`	Guía de entrega
Autoevaluación y coevaluación	Cada persona rellena su hoja	Hoja del sprint
Un asistente de IA (opcional)	Entender errores, pedir pistas y revisar si hay fugas	Con las reglas de la «Ayuda 1» del paso a paso

3 · Qué entregar

Este apartado responde a **dos preguntas distintas**, una tabla para cada una:

Tabla A · Los ficheros: qué ficheros hay, de dónde sale cada uno (ya hecho, lo modificáis, lo creáis o lo genera el cuaderno) y **cuáles se suben y adónde**.

Tabla B · Los requisitos: qué tiene que cumplir lo que entregáis y hasta dónde llegar (mínimo, estándar, extra). **Es lo que se evalúa.**

En total, al final del sprint se suben a `sprint-15/` : 6 ficheros del equipo (`registro.csv` , `03_modelos.ipynb` , `justificacion_candidato.md` , `modelo_candidato.joblib` , `metadatos.json` , `explicacion_modelo.png`) **+ 1 autoevaluación y coevaluación por persona**, es decir, **6 + tantos ficheros como personas haya en el equipo** (un equipo de 3 sube 9). «El reto» no es un fichero aparte: son esos seis ficheros. Los datos ya están en el repositorio desde el Sprint 14 y no se vuelven a subir. La *review* y la *retro* se hacen de palabra: no se suben. Si vuestro modelo guardado pesa más de 25 MB, el navegador no lo admite: mirad la «Ayuda 3».

Tabla A · Los ficheros: de dónde sale cada uno y cuáles se suben

Todo lo que se sube va al repositorio privado del equipo (`ifcd107-equipo-…`), en la carpeta **`sprint-15/`** .

Fichero	Origen	Qué hacéis vosotros	¿Se sube?
`registro.csv`	**Lo creáis a partir de una plantilla.** Os damos `registro_experimentos_plantilla.csv` , solo con la cabecera	Copiarla como `registro.csv` y añadir **una fila por prueba**, de todas las personas. Es el mismo fichero que en el repositorio del proyecto se llama `experimentos/registro.csv`	**Sí** · `sprint-15/`
`03_modelos.ipynb`	**Lo creáis.** Parte del cuaderno de *baseline* del Sprint 14 y de las funciones del cuaderno `S15_01`	Escribir el código de los modelos, la comparación, la comprobación de fugas y el test; debe ejecutarse **de arriba abajo** con semilla fija	**Sí** · `sprint-15/`
`justificacion_candidato.md`	**Lo creáis.** No hay plantilla: el paso 4 os da la tabla	Una página: tabla comparativa, coste y beneficio, por qué gana el candidato, resultado en test y, al final, la sección «Revisión con la rúbrica»	**Sí** · `sprint-15/`
`modelo_candidato.joblib` (o `.keras` / `.pt` si el candidato es la red)	**Lo genera el cuaderno** al guardar el modelo ya elegido	Guardarlo una vez elegido el candidato; no se escribe a mano	**Sí** · `sprint-15/`
`metadatos.json`	**Lo genera el cuaderno** junto al modelo	Que recoja versión, fecha, métricas y variables; no se escribe a mano	**Sí** · `sprint-15/`

Fichero	Origen	Qué hacéis vosotros	¿Se sube?
`explicacion_modelo.png`	**Lo genera el cuaderno.** Gráfico de importancia de variables, SHAP o Grad-CAM	Guardar el gráfico y escribir 2–3 frases de lectura en la justificación	**Sí** · `sprint-15/`
Autoevaluación y coevaluación (`autoevaluacion_nombre.md`)	**Ya hecha, en blanco**	Cada persona copia la plantilla con su nombre y la rellena; la coevaluación va **dentro del mismo fichero**	**Sí**, **una por persona** (no una por equipo) · `sprint-15/`
Base de datos o carpeta local de MLflow (extra)	**Lo genera** MLflow si el equipo decide probarlo	Solo si el equipo llega al nivel extra; en la práctica es opcional	No se sube
Copias intermedias de cuadernos o del registro	**Las creáis** mientras probáis	Borrarlas o dejarlas fuera de `sprint-15/`	**No se suben**

Tabla B · Los requisitos: qué tiene que cumplir y hasta dónde llegar

Requisito	Qué se pide exactamente	Nivel	Dónde se comprueba
Partición común	Todos los modelos se comparan con **la misma partición**, definida y guardada antes de probar (temporal si hay series; por grupos si hay fotos del mismo objeto)	Mínimo	`03_modelos.ipynb`
Baseline en el registro	El *baseline* repetido es **la primera fila** del registro	Mínimo	`registro.csv`
Tres modelos clásicos	Al menos 3 modelos, con media ± desviación de la validación cruzada	Mínimo	`03_modelos.ipynb` y `registro.csv`
Modelo neuronal	Al menos un modelo neuronal (o el asistente LLM/RAG en el brief 3) con la misma partición	Estándar	`03_modelos.ipynb` y `registro.csv`
Registro de experimentos	Una fila por prueba con modelo, ajustes, métrica, desviación y tiempo; **al menos una fila de cada persona**; ≥ 12 filas recomendadas	Mínimo	`registro.csv`
Ajuste de hiperparámetros	1–2 ajustes clave de cada familia, probados **solo con validación** y anotados	Estándar	`registro.csv`
Control de fugas	Lista de 5 preguntas contestada en una celda de texto antes de elegir; nada del test ha servido para decidir	Estándar	`03_modelos.ipynb` (celda «Comprobación de fugas»)
Elección del candidato	Se elige con validación (nunca con el test) y se justifica en una página: tabla comparativa y coste y beneficio (tiempo, complejidad, explicabilidad)	Mínimo	`justificacion_candidato.md`
Test una sola vez	El candidato se evalúa en el test **una única vez**, después de elegir; si el resultado no gusta, se anota y se explica, **no se vuelve a elegir**	Mínimo	`03_modelos.ipynb` y `justificacion_candidato.md`
Modelo guardado	Modelo del candidato y `metadatos.json` (versión, fecha, métricas, variables)	Mínimo	`modelo_candidato.joblib` y `metadatos.json`

Requisito	Qué se pide exactamente	Nivel	Dónde se comprueba
Primera explicación	Importancia de variables, SHAP (tabular) o Grad-CAM (imágenes), con 2–3 frases de lectura. Se profundiza en el Sprint 16	Estándar	`explicacion_modelo.png` y `justificacion_candidato.md`
Reproducibilidad	Semilla fija, versiones de las bibliotecas anotadas y cuaderno ejecutable de arriba abajo; Git con al menos 3 *commits* de personas distintas, sin claves ni datos personales	Estándar	`03_modelos.ipynb` y historial del repositorio
Uso de la IA	Declarar qué se pidió y qué se revisó, y **marcar en el registro las pruebas hechas con código generado por IA** (por ejemplo, en `notas`)	Mínimo	`justificacion_candidato.md` y `registro.csv`
Revisión y autoevaluación	Alguien que **no escribió esa parte** revisa cuaderno, registro y justificación con la rúbrica (justo debajo de estas tablas) y anota nivel y prueba en la sección «Revisión con la rúbrica» de `justificacion_candidato.md`; cada persona sube su hoja de autoevaluación y coevaluación	Estándar	`justificacion_candidato.md` y `sprint-15/`
Extras	Hipótesis sobre **por qué gana** ese modelo, contrastada con la explicación · análisis de errores o curvas del candidato · MLflow en local	Extra	`justificacion_candidato.md` y `03_modelos.ipynb`

Rúbrica del reto · cómo se puntúa y cómo se revisa

La usan **dos veces**: el equipo, para revisar su propio trabajo antes de entregar (es lo que dice la *Definition of Done*), y el docente, para puntuar. Se mira **cada criterio por separado** y se pone el nivel de la columna cuyo texto **se cumple entero**: si lo de un nivel se cumple a medias, es el nivel de abajo. **0** significa que no se ha entregado nada de ese criterio.

Orientación: lo que pide el nivel mínimo sale en un **2**, el estándar en un **3** y los extras en un **4**.

Criterio	1 · Empezado	2 · Mínimo	3 · Estándar	4 · Extra
Corrección técnica (¿funciona y se ha medido bien?)	Hay modelos, pero las particiones no coinciden, se ha mirado el test para elegir o el cuaderno no se ejecuta	*Baseline* y 3 modelos con la **misma** partición y media ± desviación; candidato elegido con validación; test evaluado **una sola vez**	Además, un modelo neuronal (o LLM/RAG), hiperparámetros ajustados solo con validación, lista de fugas contestada, modelo guardado con `metadatos.json` y una explicación del modelo	Además, análisis de errores o curvas del candidato que enseñan **dónde falla**
Reproducibilidad (¿lo puede repetir otra persona?)	No hay semilla ni versiones, o el registro está vacío o sin ajustes	Semilla fija y versiones anotadas; el registro tiene las columnas rellenas	Otra persona ejecuta `03_modelos.ipynb` de arriba abajo y **obtiene las mismas cifras**; el registro tiene ≥ 12 filas con al menos una de cada persona; hay 3 o más *commits* de personas distintas, sin claves ni datos personales	Además, todas las filas del registro llevan la huella de los datos (`datos_hash`) y la semilla, y se anota el coste de las pruebas pesadas
Análisis y comunicación al cliente (¿lo entiende quien no programa?)	La justificación solo dice «es el de mejor número»	Tabla comparativa con media ± desviación y candidato nombrado	Justificación de una página con coste y beneficio, dice si la diferencia con el siguiente modelo **puede ser ruido** y explica el resultado en test, sin jerga	Además, una hipótesis de **por qué gana** ese modelo, contrastada con la explicación, y una recomendación clara con sus límites

Criterio	1 · Empezado	2 · Mínimo	3 · Estándar	4 · Extra
IA responsable (¿se sabe qué hizo la IA y qué comprobasteis?)	No dice si se ha usado IA	Dice que se ha usado IA y marca en el registro las pruebas con código generado	Dice **qué se pidió, qué se revisó y qué se cambió**; quien escribe sabe explicar cualquier línea; no hay claves ni datos personales en el repositorio	Además, cuenta un fallo concreto de la IA (por ejemplo, una fuga que no vio o que vio sin que existiera) y un límite de lo que dice la explicación del modelo

Cómo se revisa (unos 10 minutos, antes de subir; lo hace **una persona que no escribió esa parte**):

1. **Ejecutar** `03_modelos.ipynb` de arriba abajo en un entorno limpio (por ejemplo, un Colab nuevo) y comparar las cifras con la tabla de la justificación. Anotar si algo no funciona o sale distinto.
2. **Mirar el registro**: ¿están el *baseline* en la primera fila, todas las columnas rellenas, una misma partición y al menos una fila de cada persona?
3. **Buscar fugas y trampas**: ¿se contestó la lista de fugas? ¿Aparece el test antes de elegir el candidato? ¿Hay alguna métrica «demasiado buena»? Que quien lo escribió lo explique.
4. **Leer la justificación como si fuerais el cliente**: ¿se entiende por qué gana ese modelo y si la diferencia puede ser ruido? Poner un nivel (1 a 4) en cada criterio, **con una frase de prueba** (por ejemplo: «3 · el cuaderno reproduce las cifras, pero falta la fila de Lucía en el registro»). Sin prueba escrita, el nivel no vale.
5. **Apuntar el resultado** en la sección «Revisión con la rúbrica» del final de `justificacion_candidato.md` y corregir lo más fácil de arreglar. Lo demás se anota como mejora.

Cada persona, además, se puntúa a sí misma en su hoja de autoevaluación y coevaluación con **esta misma rúbrica**.

---

4 · Guía paso a paso

Son cinco pasos, de **470 minutos en total** (130 el día 1, 200 el día 2 y 140 el día 3, junto a la píldora, la práctica, la tutoría, la entrega y la review). Las jornadas de proyecto duran **6 horas completas (250 minutos de trabajo al día)**: los pasos de esta ficha ocupan unos 200 y los **50 minutos restantes de cada día son «subir de nivel»** (pasar del mínimo al estándar con la rúbrica en la mano). Cada uno dice **qué hacéis** (la lista corta), **cómo se hace** con el detalle, **qué deberíais ver** y **cuándo está hecho**. Las ayudas (IA, errores, atascos) están al final. Si el tiempo aprieta, completad el nivel mínimo y dejad lo demás para casa.

Paso 1 · Preparar el terreno y repetir el baseline (día 1, 130 minutos)

1. Fijar la semilla, anotar las versiones y crear el registro con la plantilla.
2. Definir la partición común y escribir qué modelos se van a probar y por qué.
3. Repetir el *baseline* y anotarlo como primera fila del registro.
4. Entrenar el primer modelo clásico y anotarlo.

Cómo se hace

- **Semilla y versiones (25 minutos).** Del cuaderno `S15_01` copiad a un fichero común (`utilidades.py`) las funciones `fijar_semillas`, `registrar` y `experimento`, y que **todo el equipo use la misma**. Fijad una semilla, por ejemplo `SEMILLA = 42`, y anotad las versiones (`python --version` y `pandas.__version__`, `sklearn.__version__`) en la primera celda de `03_modelos.ipynb`. **No se cambia la semilla hasta que salga un resultado bonito.**
- **El registro.** Copiad la plantilla como `registro.csv`. Cada columna tiene su función:

Columna	Qué se anota
`id`	Un número que sube con cada prueba (1, 2, 3…)
`fecha`, `responsable`	Cuándo y **quién** hizo la prueba (clave para demostrar que participa todo el equipo)

Columna	Qué se anota
`modelo`, `hiperparametros`	Qué modelo y con qué ajustes
`datos_hash`, `semilla`	Una huella corta de los datos y la semilla usada, para saber con qué se hizo
`metrica`, `valor_cv`, `desv_cv`	La métrica que fijasteis en el Sprint 14, su media y su desviación en validación cruzada
`tiempo_s`, `notas`	Cuánto tardó y lo que habéis aprendido; aquí se marca también si el código lo generó una IA

- **Partición común y plan (20 minutos).** Definid **una** partición para todo el equipo (temporal si hay series; por grupos si hay fotos del mismo objeto), guardadla y **no la cambiéis**. Reservad el test: no se toca hasta el paso 5. Escribid en la primera celda de texto qué vais a probar y por qué (por ejemplo, regresión logística, bosque, boosting y una red). **No vale «probar 200 combinaciones y quedarse con la mejor».**
- **Baseline (30 minutos).** Repetid el modelo del Sprint 14 con la misma partición y anotadlo con `experimento(...)`: será la fila 1.
- **Primer modelo (55 minutos).** Una persona entrena el primer modelo clásico mientras las demás preparan el siguiente. Anotad la fila con el nombre de quien lo ha hecho.

Qué deberíais ver. Un `registro.csv` con al menos 3 filas (*baseline*, primer modelo y alguna prueba más) y las columnas rellenas, y el cuaderno ejecutándose de arriba abajo con las mismas cifras en dos ejecuciones seguidas (señal de que la semilla funciona). El *baseline* tiene la métrica más baja o similar a la de un modelo que no sabe nada.

Hecho cuando: hay semilla, versiones, partición común, plan escrito y el registro tiene el *baseline* y al menos un modelo más.

Paso 2 · Probar los modelos clásicos y el neuronal (día 2, 140 minutos)

1. Entrenar al menos 3 modelos clásicos con la misma partición y anotar cada prueba.
2. Entrenar al menos un modelo neuronal (o el asistente LLM/RAG en el brief 3).
3. Ajustar 1–2 hiperparámetros clave de cada familia, solo con validación.
4. Comprobar que cada persona ha anotado al menos una fila.

Cómo se hace

Un reparto posible, si sois 3 o 4:

Persona	Qué prueba	Minutos
A	Segundo y tercer modelo clásico (por ejemplo, bosque y boosting)	50
B	Modelo neuronal (o RAG/LLM en el brief 3)	60
C	Ajuste de 1–2 hiperparámetros de las familias de A (profundidad, número de árboles…)	40
D (si hay 4)	Repetir una prueba de otra persona con su código para comprobar que sale igual	40

- **Cada prueba se anota nada más terminarla**, con `experimento(...)`; no esperéis al final del día. Si tarda mucho o usa una API de pago, trabajad con un subconjunto de datos y **anotad el coste en `notas`**.
- **Ajustar sin trampa.** Probad 2 o 3 valores de un hiperparámetro y anotad **cada valor como una fila**. Anotad también lo que no mejora («empeoró al aumentar la profundidad: sobreajuste»): también es información.
- **Si una prueba la ha generado una IA,** escribidlo en `notas`.
- **Que cada persona toque algo:** si una persona se queda sin fila, que pruebe un hiperparámetro de otra y la anote con su nombre.

Qué deberíais ver. Un `registro.csv` con unas 8–12 filas de personas distintas, con valores de `valor_cv` y `desv_cv` rellenos. Si dos modelos tienen medias casi iguales y desviaciones parecidas, es normal.

Hecho cuando: hay *baseline* y 3 modelos clásicos y 1 neuronal en el registro con la misma partición, y cada persona tiene al menos una fila.

Paso 3 · Comprobar que no hay fugas y tutoría (día 2, 60 minutos)

1. Contestar la lista de fugas en una celda de texto de `03_modelos.ipynb`.
2. Corregir lo que falle y repetir las pruebas afectadas.
3. Ir a la tutoría técnica de 20 minutos con el docente.

Cómo se hace

- **La lista (20 minutos).** Copiad estas cinco preguntas a una celda de texto titulada «Comprobación de fugas» y contestad Sí / No con una frase: 1. ¿Las variables se conocerían **en el momento de decidir**? 2. ¿El escalado, la imputación y la codificación se aprenden **solo con el entrenamiento** (dentro de un `Pipeline`)? 3. ¿La partición respeta el **tiempo** o los **grupos**? 4. ¿Se ha ajustado algo mirando el test? Si sí, hay que rehacer la evaluación. 5. ¿La métrica es **demasiado buena para ser verdad**? Sospechad.
- **Si algo falla,** corregid y repetid esas pruebas; en el registro, anotad en `notas` que la fila anterior tenía una fuga.
- **Tutoría (20 minutos, el docente indica la franja).** Llevad la tabla del registro y contad **qué comparación estáis haciendo** y **qué fuga podría haber**. Anotad las respuestas.
- **El resto (20 minutos):** repasad el registro: columnas vacías, filas duplicadas y nombres de modelo distintos para el mismo modelo.

Qué deberíais ver. La celda «Comprobación de fugas» con cinco respuestas, un registro limpio y, si ha habido una fuga, una fila anotada que lo explica. Ninguna prueba con un resultado casi perfecto sin explicación.

Hecho cuando: la lista está contestada, el registro está limpio y la tutoría ha quedado anotada.

Paso 4 · Elegir el candidato sin tocar el test (día 3, 70 minutos)

1. Calcular media ± desviación de cada modelo.
2. Comparar teniendo en cuenta la incertidumbre y añadir coste y complejidad.
3. Elegir el candidato y escribir la justificación de una página.

Cómo se hace

- **Media ± desviación (15 minutos).** Con el registro, haced una tabla con una fila por modelo y la mejor prueba de cada uno. Si dos modelos se **solapan** (la diferencia entre sus medias es menor que su desviación), la diferencia puede ser **ruido**.
- **Coste y complejidad (15 minutos).** Añadid el tiempo de entrenamiento (`tiempo_s`), el tamaño o la dificultad de mantenerlo y lo fácil que es explicar su decisión.
- **Elegir (15 minutos).** Si dos modelos van casi igual, **elegid el más simple y explicadlo**. No se mira el test.
- **Escribir `justificacion_candidato.md` (25 minutos).** Una página con esta tabla (rellenad con vuestras cifras) y unas líneas debajo que digan por qué gana:

Modelo	Métrica (media ± desv.)	Tiempo	Explicabilidad	Comentario
Baseline				
Modelo A				
Modelo B				
Neuronal				

Si vais a por el nivel extra, añadid una **hipótesis de por qué gana**: por ejemplo, «los datos tienen relaciones no lineales entre antelación y canal que un árbol captura mejor que una regresión».

Qué deberíais ver. Un candidato elegido con una frase del tipo «Elegimos X (0,71 ± 0,02) frente a Y (0,72 ± 0,04): la diferencia puede ser ruido y X es 15 veces más rápido» (cifras de ejemplo).

Hecho cuando: `justificacion_candidato.md` tiene la tabla comparativa y el candidato razonado, y nadie ha mirado el test.

Paso 5 · Test una sola vez, explicación, revisión y entrega (día 3, 70 minutos)

1. Evaluar el candidato en el test, una sola vez.
2. Guardar el modelo con `metadatos.json` y hacer la primera explicación.
3. Revisar con la rúbrica y anotar el resultado.
4. Subir los ficheros a `sprint-15/`.

Cómo se hace

- **Test, una vez (10 minutos).** Ejecutad **una sola vez** la evaluación del candidato en el test y copiad el resultado a la justificación. Si no os gusta, **no volváis a elegir con él**: anotadlo y explicadlo.
- **Guardar el modelo (10 minutos).** Guardad el candidato con `joblib.dump(modelo, "modelo_candidato.joblib")` (o el formato de la red) y escribid `metadatos.json` con versión de las bibliotecas, fecha, métrica en validación y en test, y nombres de las variables. En el brief 3, guardad el asistente tal como lo ejecutáis y apuntad en `metadatos.json` el modelo de lenguaje, los parámetros y el modo de recuperar los textos.
- **Primera explicación (20 minutos).** Importancia de variables, SHAP (si los datos son tabulares) o Grad-CAM (si son imágenes). Guardad el gráfico como `explicacion_modelo.png` y escribid 2–3 frases de lectura en la justificación («la variable más importante es …, lo que tiene sentido porque …»). Se profundiza en el Sprint 16.
- **La revisión con la rúbrica (10 minutos).** Quien revisa sigue los cinco pasos de «Cómo se revisa» (apartado 3) y añade al final de `justificacion_candidato.md` una sección como esta:

```
## Revisión con la rúbrica
**Revisado por:** <nombre> · **Fecha:** <dd/mm/aaaa>

| Criterio | Nivel (1–4) | Prueba: una frase de lo que has comprobado |
|---|---|---|
| Corrección técnica | | |
| Reproducibilidad | | |
| Análisis y comunicación al cliente | | |
| IA responsable | | |

**Lo primero que mejoraríamos:** <una cosa concreta>
```

- **Subir los ficheros (la vía del curso, desde el navegador).** Seguid Cómo entregar en GitHub: (1) abrid el repositorio del equipo y **entrad primero en la carpeta `sprint-15`**; (2) *Add file* → *Upload files*; (3) arrastrad los ficheros; (4) escribid un mensaje corto y pulsad **Commit changes**. Cada subida es un *commit* con el nombre de quien la hace. Un reparto posible: la persona A sube `03_modelos.ipynb` («Añado cuaderno de modelos»), la B sube `registro.csv` y `explicacion_modelo.png`, la C sube `justificacion_candidato.md`, `modelo_candidato.joblib` y `metadatos.json`. Los cuadernos de Colab se descargan con *Archivo* → *Descargar* → *.ipynb*. Los nombres de archivo, **en minúsculas y sin espacios**.

Qué deberíais ver. En la carpeta `sprint-15` del repositorio, los seis ficheros del equipo y una hoja de autoevaluación por persona (**6 + N**); y en el historial (el icono del reloj), al menos tres *commits* con nombres distintos.

Hecho cuando: el test se ha evaluado una vez, el modelo y su explicación están guardados, la sección «Revisión con la rúbrica» está rellena y todo está en `sprint-15/`.

Después, 60 minutos de review y retro. Cada persona sube su autoevaluación y coevaluación a `sprint-15/` (en el tiempo de entrega). En la *review*, cada equipo enseña en 5 minutos: (1) la tabla de modelos con media ± desviación, (2) el candidato elegido y **por qué** (no solo la métrica), (3) una fuga que evitó o detectó, (4) una hipótesis sobre por qué gana ese modelo y (5) su registro de experimentos. El resto de equipos pregunta: *«¿cómo sabéis que la diferencia no es ruido?»*

Ayuda 1 · Cómo usar la IA en este reto

La IA puede ahorrar tiempo, pero **da código que parece funcionar y no se entiende**, y puede pasar por alto una fuga o ver una que no existe. Reglas:

- **Todo lo que dé, lo ejecutáis y lo entendéis.** Si el docente pregunta por una línea, tenéis que saber explicarla.

- **El test se usa una sola vez, con el modelo ya elegido.** Elegir por el test invalida el resultado.
- **Declaradlo** en `justificacion_candidato.md` y **marcad en el registro** las pruebas hechas con código generado por IA.
- **Nada de datos personales ni claves** en una IA ni en el repositorio.

Para qué	Ejemplo de petición	Qué comprobáis
Entender un modelo	«Explícame cuándo conviene un modelo de boosting frente a una regresión logística, con un ejemplo sencillo»	Que la explicación encaja con **vuestros datos**
Depurar un error	«Este código de scikit-learn da este error: *[error]*. Datos: *[estructura]*»	Que entendéis la causa
Detectar fugas	«Aquí está mi pipeline: *[código]*. ¿Hay alguna operación que use el test o el futuro?»	Que **verificáis** la respuesta; la IA puede pasarlas por alto o ver fugas inexistentes
Interpretar SHAP	«Tengo este gráfico SHAP (descripción). ¿Qué significa que la variable X tenga ese efecto?»	Contrastadlo con el sentido del negocio

Ayuda 2 · Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
Elegir el modelo mirando el test	«Para ver cuál es mejor»	Elegid solo con validación; el test, una vez
Resultados «increíbles» (99 %)	Fuga de datos	Revisad variables y partición con la lista del paso 3
Registro con 3 filas de una sola persona	El responsable técnico lo hace todo	Cada persona prueba algo y lo anota con su nombre
Cada persona tiene su propio `registro.csv`	Se trabaja en copias distintas del cuaderno	Juntad todas las filas en un único `registro.csv` antes de subir
Comparar con particiones distintas	Cada uno prueba por su lado	Una partición común, guardada y compartida
Mejor número = mejor modelo	Se mira solo la métrica	Añadid tiempo, simplicidad y explicabilidad
Mezclar fotos del mismo objeto entre entrenamiento y test	En imágenes es fácil	Particionad por objeto o por parcela
Cambiar la semilla hasta que salga un resultado bonito	Prisa o ganas de ganar	Una semilla para todo el equipo; si cambia mucho, el modelo es inestable y hay que decirlo

Ayuda 3 · Si os atascáis

1. **No sabemos qué modelos probar:** repasad el *leaderboard* del Sprint 8 y la red del Sprint 10, y usad las mismas familias.
2. **Dos modelos dan casi lo mismo:** perfecto; elegid el más simple y explicadlo.
3. **No tenemos GPU o va lenta:** usad un subconjunto de los datos y anotad el coste en el registro.
4. **El modelo guardado pesa más de 25 MB:** no lo subáis por el navegador; avisad al docente, subid `metadatos.json` y dejad escrito en él cómo volver a generar el modelo con vuestro cuaderno.
5. Preguntad al equipo y, después, al docente. Más ayuda: Cómo se trabaja un reto.